Multiplexores

Cristian Miguel Jiménez
20240712@itla.edu.do
MECATRONICA
ITLA
La Caleta, Santo Domingo

Resumen - En esta práctica de laboratorio se trabajó en el diseño y la implementación de circuitos lógicos combinacionales, empleando los multiplexores (MUX) como componente principal. El propósito fue desarrollar sistemas digitales capaces de generar salidas determinadas a partir de entradas específicas, evidenciando así que los multiplexores pueden utilizarse como generadores de funciones lógicas universales

Abstract - In this laboratory practice, the focus was on the design and implementation of combinational logic circuits, using multiplexers (MUX) as the main component. The goal was to create digital systems capable of producing specific outputs from given inputs, demonstrating that multiplexers can be used as universal logic function generators

INTRODUCCIÓN

En este documento se presenta el desarrollo de un circuito combinacional utilizando multiplexores 4×1. A lo largo del trabajo se muestra el proceso de diseño e implementación del circuito, así como el análisis de su funcionamiento dentro de un sistema digital.

MARCÓ TEÓRICO

Circuito combinacional:

Es un conjunto de compuertas lógicas en el que la salida depende únicamente de los valores actuales de las entradas, sin almacenar información ni considerar estados anteriores. Estos circuitos se emplean para implementar diversas funciones lógicas, como decodificadores, multiplexores, sumadores, entre otros. Su comportamiento puede representarse completamente mediante tablas de verdad y expresiones booleanas.

Multiplexor:

Un multiplexor es un circuito lógico que permite seleccionar una señal entre varias entradas y dirigirla hacia una única salida. Esta selección se realiza mediante líneas de control llamadas selectores. En esencia, funciona como un interruptor digital que determina cuál de las entradas se conecta a la salida.

- Montaje experimental

Para la realización de este laboratorio se utilizaron los siguientes componentes:

1. Resistencias de 1 kΩ
2. Project board
3. Display de 7 segmentos
4. FPGA Tang Primer 25K

Asimismo, se emplearon los siguientes programas de simulación y desarrollo:

- Multisim
- Gowin

DISEÑO DE CIRCUITO

Para comenzar con el diseño del circuito, es necesario partir de la siguiente tabla de verdad, la cual define el comportamiento del

A	B	C	D	E	F_a	F_b	F_c	F_d
0	0	0	0	0	1	1	0	0
0	0	0	0	1	1	1	1	X
0	0	0	1	0	0	0	0	0
0	0	0	1	1	0	0	0	X
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	X	0
0	0	1	1	0	0	0	1	0
0	0	1	1	1	0	0	1	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	X	X
0	1	0	1	0	0	0	0	0
0	1	0	1	1	1	1	1	X
0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	0	1	0
0	1	1	1	0	0	0	0	0
0	1	1	1	1	0	0	0	0
1	0	0	0	0	X	0	0	1
1	0	0	0	1	1	1	1	X
1	0	0	1	0	0	1	0	1
1	0	0	1	1	1	1	0	0
1	0	1	0	0	0	0	0	1
1	0	1	0	1	0	0	1	1
1	0	1	1	0	1	0	0	1
1	0	1	1	1	0	0	0	1
1	1	0	0	0	1	1	1	0
1	1	0	0	1	1	1	1	1
1	1	0	1	0	0	1	0	0
1	1	0	1	1	0	0	0	1
1	1	1	0	0	0	0	0	0
1	1	1	0	1	0	1	0	0
1	1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0	1

A esta tabla de verdad se le debe aplicar el método **VEM**, por lo que se dividirá en **cuatro grupos**. Además, se considerará que **A y B** funcionarán como las líneas **selectoras** de los multiplexores.

.

Tabla de verdad								
A	B	C	D	E	Fa	Fb	Fc	Fd
0	0	0	0	0	1	1	0	0
0	0	0	0	1	1	1	1	X
0	0	0	1	0	0	0	0	0
0	0	0	1	1	0	0	0	X
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	X	0
0	0	1	1	0	0	0	1	0
0	0	1	1	1	0	0	1	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	x	X
0	1	0	1	0	0	0	0	0
0	1	0	1	1	1	1	1	X
0	1	1	0	0	1	0	1	0
0	1	1	0	1	0	0	1	0
0	1	1	1	0	0	0	0	0
0	1	1	1	1	0	0	0	0
1	0	0	0	0	x	0	0	1
1	0	0	0	1	1	1	1	X
1	0	0	1	0	0	1	0	1
1	0	0	1	1	1	1	0	0
1	0	1	0	0	0	0	0	1
1	0	1	0	1	0	0	1	1
1	0	1	1	0	1	0	0	1
1	0	1	1	1	0	0	0	1
1	1	0	0	0	1	1	1	0
1	1	0	0	1	1	1	1	1
1	1	0	1	0	0	1	0	0
1	1	0	1	1	0	0	0	1
1	1	1	0	0	0	0	0	0
1	1	1	0	1	0	1	0	0
1	1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0	1

Partiendo de esa división, realizamos K-maps y los circuitos combinacionales para que la salida sea la esperada:

Función Fa vem:

Fa1=

A \ BC	00	01	11	10
0	1	1	0	0
1	0	0	0	0

F= C'D'

Fa2=

A \ BC	00	01	11	10
0	1	1	1	0
1	1	0	0	0

F= D'E' + C'E

Fa3=

A \ BC	00	01	11	10
0	X	1	1	0
1	0	0	0	1

F= C'E + CDE'

Fa=4

A \ BC	00	01	11	10
0	1	1	0	0
1	0	0	0	0

F= C'D'

función completa de Fa
=A'B'C'D'+A'B(D'E'+C'E)+AB'(C'E+CDE')+ABC'D'

Función Fb vem:

Fb1=

A \ BC	00	01	11	10
0	1	1	0	0
1	1	1	0	0

F= D'

Fb2=

A \ BC	00	01	11	10
0	1	1	1	0
1	0	0	0	0

F== C'D' + C'E

Fb3=

A \ BC	00	01	11	10
0	0	1	1	1
1	0	0	0	0

F= C'E+ C'D

Fb4=

A \ BC	00	01	11	10
0	1	1	0	1
1	0	1	0	0

F== C'E' + D'E

Función completa de Fb =
A'B'D'+A'B(C'D'+C'E)+AB'(C'E+C'D)+AB(C'E'+D'E)

Funcion Fc vem:

Fc1=

A \ BC	00	01	11	10
0	0	1	0	0
1	1	x	1	1

F= C + D'E

Fc2=

A \ BC	00	01	11	10
0	0	x	1	0
1	1	1	0	0

F= C'D + CD'

Fc3=

A \ BC	00	01	11	10
0	0	1	0	0
1	0	1	0	0

F= D'E

Fc4=

A \ BC	00	01	11	10
0	X	1	1	0
1	0	0	0	0

F= C'D'

Función completa de Fc =
A'B'(C+D'E)+A'B(C'D+CD')+AB'D'E+ABC'D'

Funcion Fd vem:

Fd1=

A \ BC	00	01	11	10
0	0	x	x	0
1	0	0	0	0

F= 0

Fd2=

A \ BC	00	01	11	10
0	0	x	x	0
1	0	0	0	0

F= 0

Fd3

A \ BC	00	01	11	10
0	1	x	0	1
1	1	1	1	1

F= E' + C

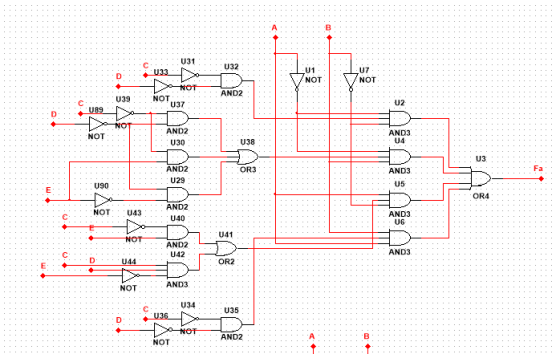
Fd4=

A \ BC	00	01	11	10
0	0	1	1	0
1	0	0	1	0

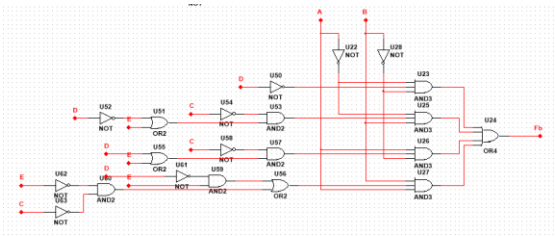
F= C'E + DE

Función completa de Fd
=A'B'+A'B+AB'(E'+C)+AB(C'E+DE)

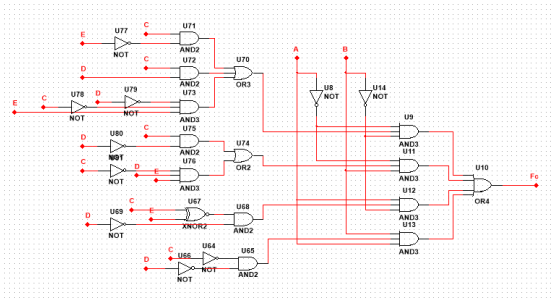
Partiendo esas funciones, realizamos las simulaciones haciendo uso de multisim:



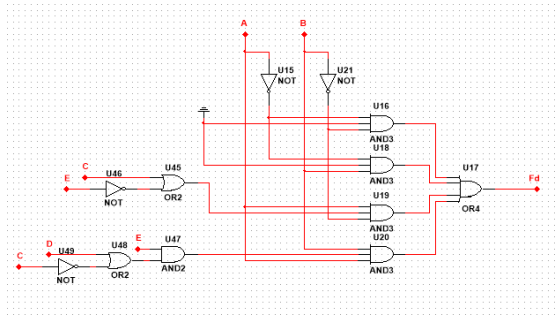
Salida Fa



Salida Fb

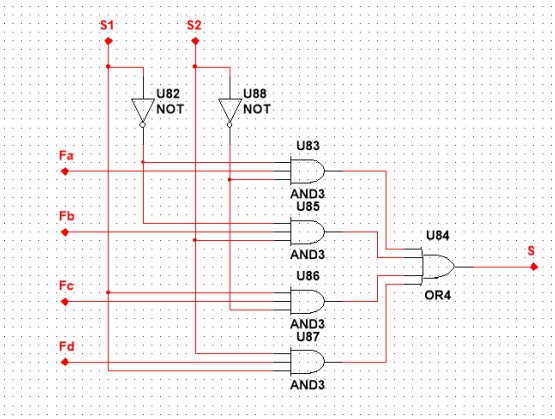


Salida Fc



Salida Fd

Y por último, hacemos un solo multiplexor, para poder seleccionar la salida querida:



MUX final

Donde, 00 es Fa, 01 es Fb, 10 es Fc y 11 es Fd.

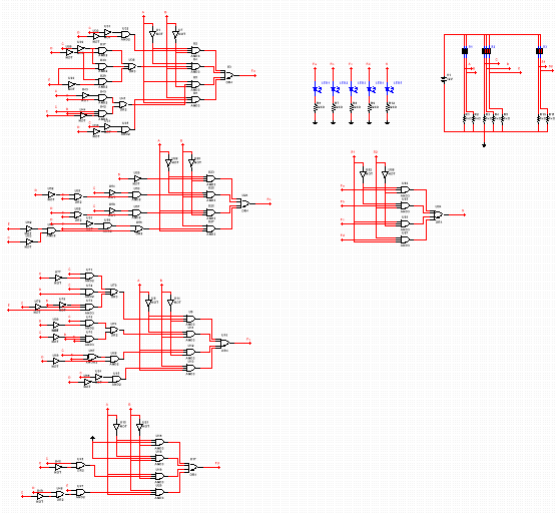


Diagrama completo

El código de un mux en verilog es el siguiente:

```
module mux4a1 (
    input wire A, B,
    input wire D0, D1, D2, D3,
    output wire Y
);

wire nA, nB ;
not U_NOTA (nA, A) ;
not U_NOTB (nB, B) ;

wire term1, term2, term3, term4 ;
and U_AND0(term1, D0, nA, nB) ;
and U_AND1(term2, D1, nA, B) ;
and U_AND2(term3, D2, A, nB) ;
and U_AND3(term4, D3, A, B) ;

or U_OR(Y, term1, term2, term3, term4) ;

endmodule
```

Mux 4 a 1

Y crearemos el código de lo anteriormente visto:

```
module mux (
    input wire A, B, C, D, E,
    input wire S1, S2 ,
    output wire Fa, Fb, Fc, Fd,
    output wire S,
    output wire a1, b1, c1, d1, e1, f1, g1,
    output wire a2, b2, c2, d2, e2, f2, g2,
    output wire a3, b3, c3, d3, e3, f3, g3,
    output wire a4, b4, c4, d4, e4, f4, g4
) ;

//Negaciones de CDE
wire nC, nD, nE ;
not U_NOTC (nC, C) ;
not U_NOTD (nD, D) ;
not U_NOTE (nE, E) ;

//Entradas de mux
wire IA0, IA1, IA2, IA3 ;
wire IB0, IB1, IB2, IB3 ;
wire IC0, IC1, IC2, IC3 ;
wire ID0, ID1, ID2, ID3 ;

//Funciones primer MUX
and U_AND00(IA0, nC, nD) ;

wire t0, t1, t2 ;
and U_AND01(t0, nD, nE) ;
and U_AND02(t1, nC, nD) ;
and U_AND03(t2, nC, E) ;
or U_OR1(IA1, t0, t1, t2) ;

wire t3, t4 ;
and U_AND04(t3, nC, E) ;
and U_AND05(t4, C, D, nE) ;
or U_OR2(IA2, t3, t4);

and U_AND06(IA3, nC, nD) ;

//Funciones Segundo MUX
buf U_BUF0 (IB0, nD) ;

wire t5, t6 ;
or U_OR3(t5, nD, E) ;
and U_AND07(IB1, t5, nC) ;

or U_OR4(t6, E, D) ;
and U_AND08(IB2, t6, nC) ;

wire t7, t8 ;
and U_AND09(t7, nC, nE) ;
and U_AND10(t8, nD, E) ;
or U_OR5(IB3, t7, t8) ;

//Funciones tercer MUX
wire l0, l1, l2 ;
and U_AND11(l0, C, nE) ;
and U_AND12(l1, C, D) ;
and U_AND13(l2, nC, nD, E) ;
or U_OR6(IC0, l0, l1, l2) ;

wire l3, l4, l5 ;
and U_AND14(l3, C, nD) ;
and U_AND15(l4, nC, nD, E) ;
or U_OR7(IC1, l3, l4) ;

xnor U_XNOR(l5, C, E) ;
and U_AND16(IC2, l5, nD) ;

and U_AND17(IC3, nC, nD) ;

//Funciones cuarto mux
buf U_BUF1(ID0, 1'b0) ;
buf U_BUF2(ID1, 1'b0) ;

or U_OR8(ID2, C, nE) ;

wire l6 ;
or U_OR9(l6, nC, D) ;
and U_AND18(ID3, l6, E) ;

//Intancias de los mux
mux4a1 U_MUXFA (
    .A (A),  .B (B),
    .D0 (IA0), .D1 (IA1), .D2 (IA2), .D3 (IA3) ,
    .Y (Fa)
);

mux4a1 U_MUXFB (
    .A (A), .B (B),
    .D0 (IB0), .D1 (IB1), .D2 (IB2), .D3 (IB3) ,
    .Y (Fb)
);

mux4a1 U_MUXFC (
    .A (A), .B (B),
    .D0 (IC0), .D1 (IC1), .D2 (IC2), .D3 (IC3) ,
    .Y (Fc)
);

mux4a1 U_MUXFD (
    .A (A), .B (B),
    .D0 (ID0), .D1 (ID1), .D2 (ID2), .D3 (ID3) ,
    .Y (Fd)
);

mux4a1 U_MUXFS (
    .A (S1), .B (S2),
    .D0 (Fa), .D1 (Fb), .D2 (Fc), .D3 (Fd) ,
    .Y (S)
);
```

Ahora, partiendo de que ya tenemos un decoder que convierte de BCD a siete segmentos, crearemos un conversor que pase un numero binario de 5 bit a BCD.

Para ello, generaremos la siguiente tabla de verdad:

Tabla de verdad													
A	B	C	D	E	A 1	B 1	C 1	D 1	A 2	B 2	C 2	D 2
0	0	0	0	0	x	x	0	0	0	0	0	0
0	0	0	0	1	x	x	0	0	0	0	0	1
0	0	0	1	0	x	x	0	0	0	0	1	0
0	0	0	1	1	x	x	0	0	0	0	1	1
0	0	1	0	0	x	x	0	0	0	1	0	0
0	0	1	0	1	x	x	0	0	0	1	0	1
0	0	1	1	0	x	x	0	0	0	1	1	0
0	0	1	1	1	x	x	0	0	0	1	1	1
0	1	0	0	0	x	x	0	0	1	0	0	0
0	1	0	0	1	x	x	0	0	1	0	0	1
0	1	0	1	0	x	x	0	1	0	0	0	0
0	1	0	1	1	x	x	0	1	0	0	0	1
0	1	1	0	0	x	x	0	1	0	0	1	0
0	1	1	0	1	x	x	0	1	0	0	1	1
0	1	1	1	0	x	x	0	1	0	1	0	0
0	1	1	1	1	x	x	0	1	0	1	0	1
1	0	0	0	0	x	x	0	1	0	1	1	0
1	0	0	0	1	x	x	0	1	0	1	1	1
1	0	0	1	0	x	x	0	1	1	0	0	0
1	0	0	1	1	x	x	0	1	1	0	0	1
1	0	1	0	0	x	x	1	0	0	0	0	0
1	0	1	0	1	x	x	1	0	0	0	0	1

1	0	1	1	0	x	x	1	0	0	0	1	0
1	0	1	1	1	x	x	1	0	0	0	1	1
1	1	0	0	0	x	x	1	0	0	1	0	0
1	1	0	0	1	x	x	1	0	0	1	0	1
1	1	0	1	0	x	x	1	0	0	1	1	0
1	1	0	1	1	x	x	1	0	0	1	1	1
1	1	1	0	0	x	x	1	0	1	0	0	0
1	1	1	0	1	x	x	1	0	1	0	0	1
1	1	1	1	0	x	x	1	1	0	0	0	0
1	1	1	1	1	x	x	1	1	0	0	0	1

Partiendo de la tabla de verdad, podemos deducir las ecuaciones:

```
module bin5tobcd (
    input wire A, B, C, D, E ,
    output wire C1, D1, A2, B2, C2, D2
) ;

//Negaciones de entradas
wire nA, nB, nC, nD, nE ;
not U_NOT0 (nA, A) ;
not U_NOT1 (nB, B) ;
not U_NOT2 (nC, C) ;
not U_NOT3 (nD, D) ;
not U_NOR4 (nE, E) ;

//Funcion de C1
wire t0, t1 ;
and U_AND00(t0, A, C) ;
and U_AND01(t1, A, B) ;
or U_OR01(C1, t0, t1) ;

//Funcion de D1
wire t2, t3, t4, t5 ;
and U_AND02(t2, A, nB, nC) ;
and U_AND03(t3, nA, B, D) ;
and U_AND04(t4, nA, B, C) ;
and U_AND05(t5, B, C, D) ;
or U_OR02(D1, t2, t3, t4, t5) ;

//Funcion de A2
wire t6, t7, t8 ;
and U_AND06(t6, nA, B, nC, nD) ;
and U_AND07(t7, A, nB, nC, D) ;
and U_AND08(t8, A, B, C, nD) ;
or U_OR03(A2, t6, t7, t8) ;

//Funcion de B2
wire t9, t10, t11, t12 ;
and U_AND09(t9, nA, nB, C) ;
and U_AND10(t10, A, nC, nD) ;
and U_AND11(t11, nA, C, D) ;
and U_AND12(t12, A, B, nC) ;
or U_OR04(B2, t9, t10, t11, t12) ;
```

```
//Funcion de C2
wire t13, t14, t15, t16, t17 ;
and U_AND13(t13, nA, nB, D) ;
and U_AND14(t14, nB, C, D) ;
and U_AND15(t15, A, nB, nC, nD) ;
and U_AND16(t16, nA, B, C, nD) ;
and U_AND17(t17, A, B, nC, D) ;
or U_OR05(C2, t13, t14, t15, t16, t17) ;

//Funcion de D2
buf U_BUF0(D2, E) ;

endmodule
```

Por otro lado, creamos una función para poder visualizar mediante un display cual es las opciones vemos en el ultimo MUX, y lo hacemos con la siguiente tabla de verdad:

Tabla de verdad								
A	B	a	b	c	d	e	f	g
0	0	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1
1	0	1	0	0	1	1	1	0
1	1	0	1	1	1	1	0	1

Y partiendo de esta tabla de verdad, realizamos el código:

```
.g (g1)
.g (g2)
(D2)
.g (g3)
.g (g4)
```

```
5  module A_to_B_7seg (
6      input wire A, B,
7      output wire a, b, c, d, e, f, g
8  );
9
10 //negacion de A y B
11 wire nA, nB ;
12 not U_NOTA(nA, A) ;
13 not U_NOTB(nB, B) ;
14
15 //Salida a
16 buf U_BUF0(a, nB) ;
17
18 //Salida b
19 xnor U_XNOR0(b, A, B) ;
20
21 //Salida c
22 or U_OR0 (c, nA, B) ;
23
24 //Salida d
25 or U_OR1 (d, A, B) ;
26
27 //Salida e
28 buf U_BUF1(e, 1'b1) ;
29
30 //Salida f
31 nand U_NAND0(f, A, B) ;
32
33 //Salida G
34 or U_OR5 (g, nA, B) ;
35
36 endmodule
```

Ya con todo lo que necesario realizado, instanciamos las funciones para poder tener los display de salida:

Problema 3: Diseñe un Decodificador de código Exceso-3 a Decimal. Para este circuito solo deben de hacer uso de un DEMUX 16x1 y las compuertas necesarias. Debajo la tabla de correspondencia.

- **Tabla de verdad**



Esta tabla muestra cómo convertir cada código Exceso-3 de 4 bits (A, B, C, D) a su equivalente decimal para usarlo con un DEMUX 1×16. En las filas tienes todas las combinaciones válidas de Exceso-3 (desde 0011 hasta 1100), y en la columna Fe aparece el número decimal que corresponde (con un "1" en la posición del decimal activo y "0" en las demás). Los "x" marcan los códigos inválidos de Exceso-3 (que no usamos). Así, al conectar esas entradas al DEMUX y usar las líneas de selección adecuadas, cada pulso activa la salida única que representa el valor decimal real de ese código.

- **K-maps**

Fa

AB \ CD	00	01	11	10
00	X	X	0	X
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

F= AB + ACD

Fb

AB \ CD	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

F= B'C' + B'D' + BCD

Fc

AB \ CD	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

F= C'D + CD'

Fd

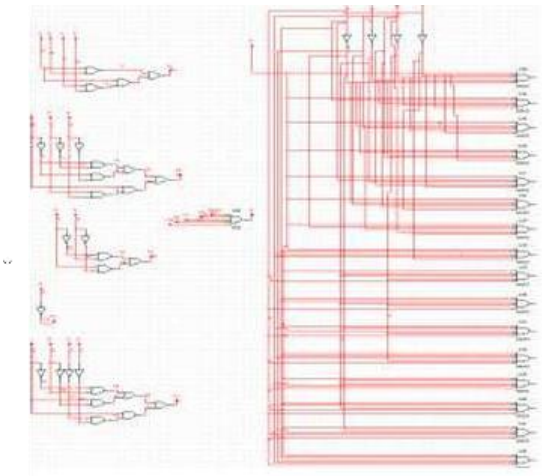
AB \ CD	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

F= D'

Fe

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	0	0	0
10	1	1	1	1

F= A'B + AB' + AC'D'



En este esquema diseñé el decodificador de código Exceso-3 a decimal usando un DEMUX 16×1 más unas compuertas lógicas para filtrar solo las combinaciones válidas. A la derecha está el DEMUX con sus 16 salidas, cada una conectada a los diodos de las compuertas AND que permiten pasar la señal solo cuando la entrada (A-D) coincide con uno de los códigos de Exceso-3 que quiero decodificar. A la izquierda ves varios grupos de compuertas NOT y AND que se encargan de generar las señales de habilitación de cada salida del DEMUX según el valor de A, B, C y D (marcando con “x” los códigos inválidos). Cuando pones en las entradas un código Exceso-3 válido, solo una de las líneas del DEMUX queda activa pasando por su AND correspondiente, y así obtengo el bit de salida decimal que buscaba.

PROBLEMA 4 Diseñe un Decodificador de código Gray a Decimal. Para este circuito solo deben de hacer uso de un DEMUX 16x1 y las compuertas necesarias.

Tabla de verdad :

A	B	C	D	Fa	Fb	Fc	Fd	Fe
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	1
0	0	1	1	0	0	1	0	1
0	1	0	0	0	1	1	1	1
0	1	0	1	0	1	1	0	1
0	1	1	0	0	1	0	0	1
0	1	1	1	0	1	0	1	1
1	0	0	0	x	x	x	x	0
1	0	0	1	x	x	x	x	0
1	0	1	0	x	x	x	x	0
1	0	1	1	x	x	x	x	0
1	1	0	0	1	0	0	0	1
1	1	0	1	1	0	0	1	1
1	1	1	0	x	x	x	x	0
1	1	1	1	x	x	x	x	0

En esta tabla puse cada código Gray válido (columnas A D) y marqué con “x” los que no se usan. Luego, para cada fila, dejo en 1 solo la salida decimal que corresponde y en 0 las demás (Fa–Fe). Así tengo claro qué línea del DEMUX debe activarse para convertir ese patrón Gray en su número decimal equivalente.

K-maps:

Fa

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	X	X
10	X	X	X	X

F= A

Fb

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	X	X
10	X	X	X	X

F= A'B

Fc

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	0	0	X	X
10	X	X	X	X

F= B'C + A'BC'

Fd

AB \ CD	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	X	X
10	X	X	X	X

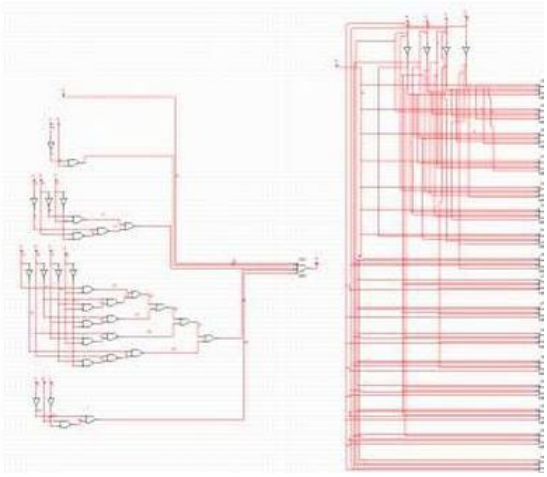
F= AD + B'C'D + B'CD' + BCD + A'BC'D'

Fe

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	X	X
10	X	X	X	X

F = A' + BC'

Multisim:



En mi proyecto de Multisim armé el decodificador de Gray a decimal usando solo un DEMUX 16×1 y unas pocas compuertas. A la izquierda puse la lógica que transforma el código Gray de 4 bits (mis entradas A-D) en los bits de selección del DEMUX, usando inversores y compuertas AND/XOR para filtrar los patrones válidos. A la derecha coloqué el DEMUX 16×1 con su entrada de datos fija en '1', así cada una de las 16 salidas se activa solo cuando los selectores coinciden con ese número. De esta forma, al ingresar cualquier código Gray válido, solo se enciende la línea decimal correcta del 0 al 15.

Conclusión

Durante esta práctica se logró implementar circuitos combinacionales usando multiplexores 4x1 para generar funciones lógicas específicas, las cuales fueron seleccionadas por un multiplexor adicional y visualizadas en un display. También se construyó un conversor binario a BCD para mostrar el valor decimal correspondiente en siete segmentos. La implementación física con FPGA y la simulación en Gowin permitieron comprobar el correcto funcionamiento de cada módulo. Esta experiencia permitió reforzar el uso de multiplexores en el diseño digital y cómo integrarlos en circuitos más complejos junto con decodificadores.